CSC3002: Functions and Libraries — Part 1

1. Functions: Core Idea

A **function** is a named, reusable block of code that performs a specific task. A function may:

- receive input through **arguments**;
- store those inputs in **parameters**;
- execute statements; and
- return a value to its caller.

```
double f(double x) {
    return x * x + 1;
}
double y = 1.0;
double z = f(y);
```

For `f(y)`, `x` is the **parameter** declared by the function, while `y` is the **argument** supplied by the caller.

Why functions are useful

- **Reuse:** write an operation once and call it many times.
- **Abstraction:** replace a sequence of low-level operations with a meaningful name.
- **Decomposition:** divide a large program into manageable components.
- **Maintenance:** change a function's implementation without changing its callers, provided its interface stays the same.
- **Top-down design / stepwise refinement:** begin with the overall problem and progressively divide it into smaller tasks.

2. Defining and Declaring Functions

General form:

```
return_type function_name(parameter_list) {
    // function body
    return expression;
}
```

- The **return type** specifies the type of result produced.
- The **function name** identifies the operation.
- The **parameter list** declares the inputs.
- The **body** implements the algorithm.

Special categories

```
bool isEven(int n) {       // Predicate function: f that returns boolean
    return n % 2 == 0;
}

void printMessage() {      // Procedure: no return value
    std::cout << "Hello";
}
```

A function returning `bool` is often called a **predicate**. A function returning no value uses the type `void` and is often called a **procedure**.

Function prototypes

A function must be declared before it is called. A **prototype** is the function header followed by a semicolon:

```
int gcd(int x, int y);
```

Prototypes let the compiler check the number and types of arguments before seeing the complete definition. Defining every helper before it is called can avoid prototypes, but usually conflicts with readable top-down organization and cannot handle cases such as

mutual recursion cleanly.

3. Functions and Algorithm Choice

Functions provide a structure for expressing algorithms, but two functions that solve the same problem can have very different efficiency.

Brute-force greatest common divisor

```
int gcd(int x, int y) {
    int guess = (x < y) ? x : y;
    while (x % guess != 0 || y % guess != 0) {
        guess--;
    }
    return guess;
}
```

This tests possible divisors from the smaller input downward. It terminates for positive integers because it will eventually reach `1`, but it may require nearly as many checks as the smaller input.

Euclid's algorithm

```
int gcd(int x, int y) {
    int r = x % y;
    while (r != 0) {
        x = y;
        y = r;
        r = x % y;
    }
    return y;
}
```

Key identity:

> `gcd(x, y) = gcd(y, x % y)`

For `gcd(78, 33)`, the remainder sequence is `12 → 9 → 3 → 0`, so the answer is `3`. For `gcd(1,000,005, 1,000,000)`, Euclid's algorithm needs only two remainder steps, compared with almost one million checks using brute force.

4. Increment Operators

Expression	Meaning	Value of expression if `i` starts at 10	Final `i`
`++i`	Pre-increment: increment first	11	11
`i++`	Post-increment: use old value first	10	11

Example with `A = {10, 11, ..., 19}`:

```
for (int i = 0; i < 9;)
    std::cout << A[i++] << ',';  // A[0] through A[8]: 10 through 18

for (int i = 0; i < 9;)
    std::cout << A[++i] << ',';  // A[1] through A[9]: 11 through 19
```

5. Function Overloading and Signatures

Function overloading allows multiple functions to use the same name when their parameter lists differ.

```
int add1(int x);
double add1(double x);
```

A function's **signature** is determined by the number and types of its parameters—not by parameter names. The compiler selects the overload whose parameters best match the supplied arguments.

In the lecture exercise, input is stored in a `double`, so both input text `1` and `1.0` call `add1(double)`. The result of `add1(x) / 4` is `0.5` in both cases.

6. Default Arguments

Default arguments make trailing parameters optional:

```
void setInitialLocation(double x = 0, double y = 0);
```

Rules:

1. Put default values in the **prototype**, not again in the definition.
2. Optional parameters must come at the end of the parameter list.
3. Once one parameter has a default, every parameter after it must also have a default.

If a function should accept either **two arguments or none**, but never exactly one, use overloading:

```
void setInitialLocation(double x, double y);

void setInitialLocation() {
    setInitialLocation(0, 0);
}
```

7. Decomposition Example: Combinations and Factorials

The mathematical formula is:

$(_{nk})=n!k!(n-k)!$

Decompose the calculation into smaller functions:

```
int fact(int n) {
    int result = 1;
    for (int i = 1; i <= n; i++) {
        result *= i;
    }
    return result;
}

int combinations(int n, int k) {
    return fact(n) / (fact(k) * fact(n - k));
}
```

This illustrates top-down design: `combinations` expresses the high-level formula, while `fact` handles the lower-level repeated multiplication.

8. Mechanics of a Function Call

When a function is called:

1. The caller evaluates each argument expression.
2. C++ copies argument values into corresponding parameter variables in a new **stack frame**.
3. The function executes using that frame for parameters and local variables.
4. A `return` statement computes the returned value and replaces the function-call expression with it.
5. The called function's stack frame is discarded, and execution resumes in the caller.

This default behavior is **call by value**. Changes to value parameters affect only local copies.

```
void swap(int x, int y) {
    int tmp = x;
    x = y;
    y = tmp;
}
```

Calling `swapWrong(n1, n2)` does not change `n1` or `n2` because only their copied values are exchanged.

9. References and Call by Reference

A **reference** is an alias for an existing object, not a copy:

```
int n1 = 1;
int& x = n1;
```

Important properties:

- A reference must be initialized when declared.
- After initialization, it cannot be rebound to another object.
- The original name and the reference name access the same object.

Reference parameters

```
void swap(int& x, int& y) {
    int tmp = x;
    x = y;
    y = tmp;
}
```

Here, `x` and `y` refer directly to the caller's variables, so the swap persists after the function returns. The arguments must be assignable objects, such as variables—not constants or temporary values.

Reference parameters have two main purposes:

- allow information to flow both into and out of a function, including returning multiple results;
- avoid copying large objects, which can improve efficiency.

Example returning two quadratic roots through references:

```
void solveQuadratic(double a, double b, double c,
                    double& r1, double& r2) {
    double disc = b * b - 4 * a * c;
    double sqrtDisc = sqrt(disc);
    r1 = (-b + sqrtDisc) / (2 * a);
    r2 = (-b - sqrtDisc) / (2 * a);
}
```

The lecture describes references as safer and less powerful than C-style pointers. Conceptually, treat a reference as another name for an existing object, even though a compiler may implement it using an address internally.

10. Operators and Operands

Common arithmetic operators include:

Operator	Operation
`+`	Addition
`-`	Subtraction or unary negation
`*`	Multiplication
`/`	Division
`%`	Remainder

An **operand** is a value or sub-expression on which an operator acts. A **binary operator** has two operands (`a + b`), while a **unary operator** has one (`-x` , `++x`).

C++ permits operator overloading for user-defined types:

```
struct Point {
    double x{};
    double y{};
};

Point operator+(Point a, const Point& b) {
    a.x += b.x;
    a.y += b.y;
    return a;
}
```

```
Point operator++(Point& p) {  // Prefix increment form
    ++p.x;
    ++p.y;
    return p;
}
```

The slides end with an implementation exercise asking for a `Point` type split across a header, implementation file, and demonstration program, with overloaded arithmetic, increment/decrement, comparison, compound-assignment, and stream-output operators.

> C++ note: some operators have restrictions. In particular, `operator=` must be a non-static member function, so it cannot be implemented as a free function.

Quick Review Checklist

You should be able to:

- distinguish a parameter from an argument;
- write a function definition and a prototype;
- explain why decomposition and abstraction improve programs;
- compare brute force with Euclid's GCD algorithm;
- predict the behavior of pre- and post-increment;
- identify a function's signature and select an overload;
- apply the rules for default arguments;
- trace a call using argument evaluation, stack frames, return values, and frame removal;
- explain why call-by-value cannot swap the caller's variables;
- use reference parameters to modify caller data or produce multiple outputs; and
- distinguish unary and binary operators and explain operator overloading.

Common Pitfalls

- Confusing a **parameter** (in a declaration/definition) with an **argument** (in a call).
- Calling a function before it has been declared.
- Expecting changes to value parameters to affect caller variables.
- Forgetting `&` in a reference parameter.
- Passing a constant to a non-const reference parameter.
- Putting a required parameter after one with a default value.
- Repeating default values in both a prototype and a definition.
- Assuming return type or parameter names distinguish overloaded functions.
- Mixing up the returned values of `++i` and `i++`.
- Choosing a correct but unnecessarily slow brute-force algorithm.

Source: CSC3002 C/C++ Programming, “Functions and Libraries,” Part 1, 35 slides.